



Notebook - Maratona de Programação

Se eu não ler o nome completo, inclusive as onomatopeias, ...

Contents		
1 Grafos	2	
1.1 Dijkstra	2	
1.2 Bfs	2	
1.3 Floyd Warshall	2	
1.4 Kruskal	2	
1.5 Bellman Ford	3	
1.6 Dfs	3	
2 Algoritmos	3	
2.1 Busca Binaria	3	
3 DP	3	
3.1 Knapsack01	3	
3.2 Longest Increasing Subsequence	4	
3.3 Longest Common Subsequence	5	
3.4 Knapsack Possible Sums	5	
3.5 Edit Distance	5	
4 Math	6	
4.1 Totient	6	
4.2 Exp	6	
4.3 Crivo	7	
4.4 Inverso Modular	7	
4.5 Fast Exponentiation	7	
4.6 Matrix Exponentiation	7	
4.7 Gcd	8	
5 ED	8	
5.1 Segtree Recursiva Lazy	8	
5.2 Segtree Iterativa	8	
5.3 Dsu	9	
6 Misc	9	
6.1 Bitwise	9	
6.2 Template	9	
6.3 Math Misc	9	
6.4 Cpp Functions	10	
6.5 Outros Notebooks	10	

1 Grafos

1.1 Dijkstra

```
//O((V + E) * log(V))
//Gera um array com as distancias minimas dos vertices ao no inicial
//Salvar grafo como lista de adjacencias
//Os pesos devem ser colocados na fila de prioridade como -w, pra serem
//ordenados da maior para a menor distancia
//As arestas sao salvas como {b, -w}, de forma que existe um caminho de a para
//b com peso w
//Eh necessario colocar o elemento {b, -w} em adj[a], e {a, -w} em adj[b]
```

```
const int oo = 1e9;
int n;
vector<pair<int, int>> adj[MAXN];
vector<int> distance(MAXN, oo);
```

```
void dijkstra(int start){
    priority_queue<pair<int, int>> pq;
    //para salvar diretamente os pesos como w, eh preciso mudar a pq para uma
    //min-heap
    //priority_queue<pii, vector<pii>, greater<pii>> pq;
    distance[start] = 0;
    pq.push({0, start});

    while(!pq.empty()){
        auto [dist, u] = pq.top(); pq.pop();

        if(-dist > distance[u]) continue;

        for(auto [b, w] : adj[u]){
            if(distance[u] + w < distance[b]){
                distance[b] = distance[u] + w;
                pq.push({-distance[b], b});
            }
        }
    }
}
```

1.2 Bfs

```
//O(V + E)
//Salvar o grafo como lista de adjacencias
```

```
vector<int> adj[n];
vector<int> visited(n);
vector<int> distance(n);
```

```
queue<int> q;
```

```
visited[start] = true;
distance[start] = 0;
```

```
q.push(start);
```

```
while(!q.empty()){
```

```
    int u = q.top(); q.pop();
```

```
    for(auto s : adj[u]){
        if(visited[s]) continue;
        visited[s] = true;
        distance[s] = distance[u] + 1;
        q.push(s);
    }
}
```

1.3 Floyd Warshall

```
//O(V^3)
//Calcula as menores distancias entre quaisquer dois
//vertices do grafo
//Inicializa com uma matriz de adjacencias adj[][],
//em que adj[a][b] guarda o peso da aresta entre a e b
//e constroi a matriz de adjacencias dist[][]
```

```
int n;
int dist[MAXN][MAXN];
int adj[MAXN][MAXN];
```

```
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        if(i == j) dist[i][j] = 0;
        else if(adj[i][j]) dist[i][j] = adj[i][j];
        else dist[i][j] = oo;
    }
}

for(int k = 0; k < n; k++){
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
        }
    }
}
```

```
//ciclos negativos
bool negative_cycle(){
    for(int i = 0; i < n; i++){
        if(dist[i][i] < 0) return true;
    }
    return false;
}
```

1.4 Kruskal

```
//O(E * log(E))
//Retorna uma MST e seu custo
//Salvar grafo como lista de adjacencias {w, a, b}
```

```
vector<tuple<int, int, int>> adj;
```

```
pair<int, vector<tuple<int, int, int>>> kruskal(int n){
    DSU dsu(n);
    sort(all(adj));
    int cost = 0;
```

```
    vector<tuple<int, int, int>> mst;
    for(auto [w, a, b] : adj){
```

```

    if(dsu.find(a) != dsu.find(b)){
        mst.push_back({w, a, b});
        cost += w;
        dsu.join(a, b);
    }
}

return {cost, mst};
}

```

1.5 Bellman Ford

```

//O(V * E)
//Calcula as distancia dos menores caminhos do
//vertice x aos demais
//Salvar o grafo como lista de arestas (a, b, w)
//Lida com ciclos negativos, enquanto Dijkstra nao
//Handbook

int n;
vector<tuple<int, int, int>> edges;
vector<int> distance(n, oo);

distance[x] = 0;
for(int i = 0; i < n - 1; i++){
    for(auto [a, b, w] : edges){
        if(distance[a] != oo)
            distance[b] = min(distance[b], distance[a] + w);
    }
}

//Detectar ciclo negativo
bool has_negative_cycle = false;
for(auto [a, b, w] : edges){
    if(distance[a] != oo && distance[a] + w < distance[b]){
        has_negative_cycle = true;
        break;
    }
}

```

1.6 Dfs

```

//O(V + E)
//Salvar o grafo como lista de adjacencias

vector<int> adj[n];
vector<bool> visited(n);

void dfs(int u){
    if(visited[u]) return;

    visited[u] = true;

    for(auto s : adj[u]){
        dfs(s);
    }
}

```

2 Algoritmos

2.1 Busca Binaria

```

ll l, r, res = -1;

while(l <= r){
    ll mid = (l + r)/2;
    if(ok(mid)){
        res = mid;
        r = mid - 1;
    }
    else{
        l = mid + 1;
    }
}

cout << res << endl;

//first true

while(l <= r){
    int mid = (l + r) / 2;
    if(check(mid)){
        r = mid - 1;
    }else{
        l = mid + 1;
    }
}

cout << l;

// last true

while(l <= r){
    int mid = (l + r) / 2;
    if(check(mid)){
        l = mid + 1;
    }else{
        r = mid - 1;
    }
}

cout << r;

```

3 DP

3.1 Knapsack01

```

//O(n * W), em que n eh o numero de itens e W eh o peso maximo
//calcula a maior pontuacao possivel ao adicionar itens na mochila,
//em que a soma dos pesos escolhidos eh no maximo W
//dp[i][ainda_pode] = maior pontuacao utilizando itens do indice 0
//ao indice i, utilizando no maximo ainda_pode de peso

struct Item{

```

```

    int w, v;
};

//recursivo com backtracking
int n;
Item items[MAXN];
vector<vector<int>> dp(MAXN, vector<int>(MAXW, -1));

int rec(int i, int ainda_pode){
    if(i >= n){
        return 0;
    }

    if(dp[i][ainda_pode] != -1){
        return dp[i][ainda_pode];
    }

    int pega = 0;
    int npega = rec(i + 1, ainda_pode);

    if(ainda_pode - items[i].w >= 0) pega = items[i].v + rec(i + 1, ainda_pode - items[i].w);

    return dp[i][ainda_pode] = max(pega, npega);
}

cout << rec(0, w) << endl;

vector<Item> backtrack(int i, int ainda_pode){
    if(i >= n) return {};

    int npega = rec(i + 1, ainda_pode);
    int pega = 0;
    if(ainda_pode - items[i].w >= 0)
        pega = items[i].v + rec(i + 1, ainda_pode - items[i].w);

    if(pega > npega){
        vector<Item> result = backtrack(i + 1, ainda_pode - items[i].w);
        result.push_back(items[i]);
        return result;
    }
    return backtrack(i + 1, ainda_pode);
}

vector<Item> chosen = backtrack(0, W);

//iterativo 1D

int n;
Item items[MAXN];
vector<int> dp(W + 1, 0);
for(int i = 0; i < n; i++){
    for(int j = W; j >= items[i].w; j--){
        dp[j] = max(dp[j], dp[j - items[i].w] + items[i].v);
    }
}

//iterativo com backtracking
int n;

```

```

Item items[MAXN];
vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

for(int i = 1; i <= n; i++){
    for(int j = 0; j <= W; j++){
        dp[i][j] = dp[i-1][j];
        if(j >= items[i-1].w)
            dp[i][j] = max(dp[i][j], dp[i-1][j - items[i-1].w] + items[i-1].v);
    }
}

vector<Item> chosen;
int j = W;
for(int i = n; i >= 1; i--){
    if(dp[i][j] != dp[i-1][j]){
        chosen.push_back(items[i-1]);
        j -= items[i-1].w;
    }
}
}

3.2 Longest Increasing Subsequence

//O(n * log(n))
//Calcula o tamanho da maior subsequencia crescente em um array
//lower_bound() calcula uma LIS estritamente crescente
//para uma LIS nao-decrescente, trocar por upper_bound()

int lis(vector<int> &arr){
    vector<int> dp;

    for(int i = 0; i < n; i++){
        int l = lower_bound(dp.begin(), dp.end(), arr[i]) - dp.begin();

        if(l == dp.size()){
            dp.push_back(arr[i]);
        }
        else{
            dp[l] = arr[i];
        }
    }

    return dp.size();
}

```

```

//LIS com backtracking

vector<int> lis_backtrack(vector<int> &arr){
    int n = arr.size();
    vector<int> dp;
    vector<int> pos(n);
    vector<int> prev(n, -1);
    vector<int> last(n, -1); // last[l] = indice mais recente com pos = l

    for(int i = 0; i < n; i++){
        int l = lower_bound(dp.begin(), dp.end(), arr[i]) - dp.begin();

        if(l == dp.size()) dp.push_back(arr[i]);
        else dp[l] = arr[i];
    }
}

```

```

        pos[i] = 1;
        if(l > 0) prev[i] = last[l - 1];
        last[l] = i;
    }

    // óreconstri seguindo os prev
    vector<int> result;
    for(int i = last[dp.size() - 1]; i != -1; i = prev[i])
        result.push_back(arr[i]);

    reverse(result.begin(), result.end());
    return result;
}

```

3.3 Longest Common Subsequence

```

//O(n * m)
//Calcula o tamanho da maior subseguencia comum entre dois arrays

int a[1001], b[1001];
int dp[1001][1001];

//recursiva
int lcs(int i, int j){
    if(i == 0 || j == 0){
        return 0;
    }

    if(dp[i][j] != -1) return dp[i][j];
    if(a[i - 1] == b[j - 1]){
        return dp[i][j] = lcs(i - 1, j - 1) + 1;
    }
    else{
        return dp[i][j] = max(lcs(i - 1, j), lcs(i, j - 1));
    }
}

//iterativa
for(int i = 0; i <= n; i++){
    dp[i][0] = 0;
}
for(int i = 0; i <= m; i++){
    dp[0][i] = 0;
}

for(int i = 1; i <= n; i++){
    for(int j = 1; j <= m; j++){
        if(a[i - 1] == b[j - 1]){
            dp[i][j] = dp[i - 1][j - 1] + 1;
        }
        else{
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
}

cout << dp[n][m] << endl;

```

```

//backtracking
vector<int> lcs_result;
int i = n, j = m;

while(i > 0 && j > 0){
    if(a[i-1] == b[j-1]){
        lcs_result.push_back(a[i-1]);
        i--;
        j--;
    }
    else if(dp[i-1][j] >= dp[i][j-1]){
        i--;
    }
    else{
        j--;
    }
}

reverse(lcs_result.begin(), lcs_result.end());

```

3.4 Knapsack Possible Sums

```

//O(n * W), em que n eh o numero de itens e W eh o peso maximo
//calcula todas as possiveis somas utilizando um dado conjunto de pesos/moedas
//total = soma de todos os pesos

int n;
vector<bool> possible(MAXSUM);
vector<int> from(MAXSUM);
vector<int> arr[MAXN];

for(int i = 0; i < n; i++){
    for(int x = total; x >= 0; x--){
        if(possible[x] && !possible[x + arr[i]]){
            possible[x + arr[i]] = true;
            from[x + arr[i]] = i;
        }
    }
}

//reconstruindo a solucao

int now = target;
while(now > 0){
    cout << from[now] << " ";
    now -= (arr[from[now]]);
}

cout << endl;

```

3.5 Edit Distance

```

//O(|a| * |b|)
//Calcula quantas operacoes sao necessarias para transformar a string A na
    string B
//Operacoes permitidas:
// *Insercao
// *Remocao

```

```

// *Substituicao

//iterativa
int dp[MAXN][MAXN];

int dist(string a, string b){
    int n = a.size();
    int m = b.size();

    for (int i=0;i<=n;i++) {
        dp[i][0] = i;
    }
    for (int j=0;j<=m;j++) {
        dp[0][j] = j;
    }

    for (int i=1;i<=n;i++) {
        for (int j=1;j<=m;j++) {
            int cost = a[i-1] != b[j-1];
            dp[i][j] = min({dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+cost});
        }
    }
    return dp[n][m];
}

cout << dist(a,b) << endl;

//recursiva
vector<vector<int>> dp(MAXN, vector<int>(MAXN, -1));

string a, b;

int dist(int i, int j){
    if(i == -1) return j + 1;
    else if(j == -1) return i + 1;
    if(dp[i][j] != -1) return dp[i][j];

    int cost = (a[i] == b[j] ? 0 : 1);

    return dp[i][j] = min({dist(i - 1, j) + 1, dist(i, j - 1) + 1, dist(i - 1,
        j - 1) + cost});
}

cout << dist(a.length() - 1, b.length() - 1) << endl;

```

4 Math

4.1 Totient

```

// O(sqrt(m))
// phi(p^k) = (p^(k-1))*(p-1) com p primo
ll phi(ll m){
    ll res = m;
    for(ll d=2;d<=m;d++){
        if(m % d == 0){
            res = (res/d)*(d-1);
            while(m%d == 0)

```

```

        m /= d;
    }
}
if(m > 1) {
    res /= m;
    res *= (m-1);
}
return res;
}

// modificacao do crivo, O(n*log(log(n)))
vector<ll> phi_to_n(ll n){
    vector<bool> isprime(n+1, true);
    vector<ll> tot(n+1);
    tot[0] = 0; tot[1] = 1;
    for(ll i=1;i<=n; i++){
        tot[i] = i;
    }

    for(ll p=2;p<=n;p++){
        if(isprime[p]){
            tot[p] = p-1;
            for(ll i=p+p;i<=n;i+=p){
                isprime[i] = false;
                tot[i] = (tot[i]/p)*(p-1);
            }
        }
    }
    return tot;
}

```

4.2 Exp

```

#include <bits/stdc++.h>
#define fastio ios::sync_with_stdio(0); cin.tie(nullptr); cout.tie(nullptr);
#define ll long long
#define endl '\n'

using namespace std;

ll fexp(ll a, ll b, ll MOD){
    ll result = 1;

    while(b > 0){
        if(b % 2 == 1) result = (result * a) % MOD;
        a = (a * a) % MOD;
        b /= 2;
    }

    return result;
}

int main(){fastio
    int t; cin >> t;

    while(t--){
        ll a, b, c; cin >> a >> b >> c;
        cout << expo(a, expo(b, c, 1000000006LL))

```

```

    }

    return 0;
}

```

4.3 Crivo

```

//O(n * log(log(n)))
//se sieve[n] == n, entao n eh primo
//para encontrar os divisores primos de n, basta fazer n /= sieve[n] eat n == 1

```

```

vector<int> sieve(N + 1, 0);
sieve[0] = 1, sieve[1] = 1;

for(int i = 2; i <= N; i++){
    if(!sieve[i]){
        sieve[i] = i;
        for(int j = i * i; j <= N; j += i){
            if(!sieve[j]) sieve[j] = i;
        }
    }
}

```

4.4 Inverso Modular

```

//Inverso modular para contas do tipo b / a % m
//Eh necessario que gcd(a, m) = 1
//Sendo phi(m) a funcao totiente de euler
//Se m for primo, entao phi(m) = m - 1
int inv(int a, int phim){
    int e = phim - 1;
    return fexp(a, e);
}

```

4.5 Fast Exponentiation

```

//O(log(b))
//retorna a^b % MOD

//recursivo
const ll MOD = 1000000007LL;

ll fexp(ll a, ll b){
    if(b == 0) return 1;

    ll half = fexp(a, b/2);
    half = (half * half) % MOD;

    if(b % 2 == 0) return half;
    return (half * a) % MOD;
}

//iterativo
const ll MOD = 1000000007LL;

ll fexp(ll a, ll b){

```

```

ll result = 1;

while(b > 0){
    if(b % 2 == 1) result = (result * a) % MOD;
    a = (a * a) % MOD;
    b /= 2;
}

return result;
}

```

4.6 Matrix Exponentiation

```

struct Matrix {
    vector<vl> m;
    int r, c;

    Matrix(vector<vl> mat) {
        m = mat;
        r = mat.size();
        c = mat[0].size();
    }

    Matrix(int row, int col, bool ident=false) {
        r = row; c = col;
        m = vector<vl>(r, vl(c, 0));
        if(ident) {
            for(int i = 0; i < min(r, c); i++) {
                m[i][i] = 1;
            }
        }
    }

    Matrix operator*(const Matrix &o) const {
        assert(c == o.r); // garantir que da pra multiplicar
        vector<vl> res(r, vl(o.c, 0));

        for(int i = 0; i < r; i++) {
            for(int k = 0; k < c; k++) {
                for(int j = 0; j < o.c; j++) {
                    res[i][j] = (res[i][j] + m[i][k]*o.m[k][j]) % MOD;
                }
            }
        }

        return Matrix(res);
    }
};

Matrix fexp(Matrix b, int e, int n) {
    if(e == 0) return Matrix(n, n, true); // identidade
    Matrix res = fexp(b, e/2, n);
    res = (res * res);
    if(e%2) res = (res * b);

    return res;
}

```

4.7 Gcd

```
//log(min(a, b))
//lcm(a, b) = a * b / gcd(a, b)
//c++ ja tem __gcd(a, b) implementado
```

```
ll gcd(ll a, ll b){
    if(b == 0) return a;

    return gcd(b, a % b);
}
```

5 ED

5.1 Segtree Recursiva Lazy

```
//Realiza consultas e §atualizaes em um índice em O(log(n))
//0-Indexed
//Valores iniciais sao salvos em (seg[n], ..., seg[2*n - 1])
//Query: retorna a operacao f aplicada no intervalo [l, r]
//Update: substitui o valor em k para x

const int oo = INT_MAX;

struct Segtree {
    int n;
    vector<int> tree, lazy;

    Segtree(int n) : n(n), tree(4*n, oo), lazy(4*n, 0) {}

    int f(int a, int b) {
        return min(a, b);
    }

    void push(int node) {
        if (lazy[node]) {
            tree[2*node] += lazy[node]; lazy[2*node] += lazy[node];
            tree[2*node+1] += lazy[node]; lazy[2*node+1] += lazy[node];
            lazy[node] = 0;
        }
    }

    int query(int node, int l, int r, int ql, int qr) {
        if (ql > r || qr < l) return oo;
        if (ql <= l && r <= qr) return tree[node];
        push(node);
        int mid = (l + r) / 2;
        return f(query(2*node, l, mid, ql, qr),
            query(2*node+1, mid+1, r, ql, qr));
    }

    void update(int node, int l, int r, int ul, int ur, int val) {
        if (ul > r || ur < l) return;
        if (ul <= l && r <= ur) { tree[node] += val; lazy[node] += val; return
; }
        push(node);
        int mid = (l + r) / 2;
```

```
        update(2*node, l, mid, ul, ur, val);
        update(2*node+1, mid+1, r, ul, ur, val);
        tree[node] = f(tree[2*node], tree[2*node+1]);
    }

    int query(int l, int r) { return query(1, 0, n-1, l, r); }
    void update(int l, int r, int val) { update(1, 0, n-1, l, r, val); }
};

// Para ler os dados iniciais, usar:
for(int i = 0; i < n; i++) seg.update(i, i, valor);
```

5.2 Segtree Iterativa

```
//Realiza consultas e §atualizaes em um índice em O(log(n))
//0-Indexed
//Valores iniciais sao salvos em (seg[n], ..., seg[2*n - 1])
//Query: retorna a operacao f aplicada no intervalo [l, r]
//Update: substitui o valor em k para x
```

```
const int oo = INT_MAX;

struct Segtree {
    int n;
    vector<int> tree;

    Segtree(int n) : n(n), tree(2*n, oo) {}

    int f(int a, int b){
        return min(a, b);
    }

    void build(){
        for(int i = n - 1; i >= 1; i--){
            tree[i] = f(tree[2*i], tree[2*i+1]);
        }
    }

    int query(int l, int r){
        l += n; r += n;
        int s = oo;
        while(l <= r){
            if(l%2 == 1) s = f(s, tree[l++]);
            if(r%2 == 0) s = f(s, tree[r--]);
            l /= 2; r /= 2;
        }
        return s;
    }

    void update(int k, int x){
        k += n;
        tree[k] = x;
        for(k /= 2; k >= 1; k /= 2){
            tree[k] = f(tree[2*k], tree[2*k+1]);
        }
    }
};
```



```
//Para ler os dados iniciais da árvore, usar:
for(int i = 0; i < n; i++){
    cin >> tree[i + n];
}
```

5.3 Dsu

```
struct DSU {
    int n;
    vector<int> parent, size;

    DSU(int n): n(n) {
        parent.resize(n, 0);
        size.assign(n, 1);

        for(int i=0;i<n;i++)
            parent[i] = i;
    }

    int find(int a) {
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }

    void join(int a, int b) {
        a = find(a); b = find(b);
        if(a != b) {
            if(size[a] < size[b]) swap(a, b);
            parent[b] = a;
            size[a] += size[b];
        }
    }
};
```

6 Misc

6.1 Bitwise

```
/ Least significant bit (lsb)
int lsb(int x) { return x&-x; }
int lsb(int x) { return __builtin_ctz(x); } // bit position
// Most significant bit (msb)
int msb(int x) { return 32-1-__builtin_clz(x); } // bit position

// Power of two
bool isPowerOfTwo(int x){ return x && (!(x&(x-1))); }

// floor(log2(x))
int flog2(int x) { return 32-1-__builtin_clz(x); }
int flog2ll(ll x) { return 64-1-__builtin_clzll(x); }

// Built-in functions
// Number of bits 1
__builtin_popcount()
__builtin_popcountll()
```

```
// Number of leading zeros
__builtin_clz()
__builtin_clzll()

// Number of trailing zeros
__builtin_ctz()
__builtin_ctzll()

//parity number of ones
__builtin_parity( x)

//set representation
for (int i = 0; i < 32; i++) {
    if (x&(1<<i)) cout << i << " ";
}

/* set operations
intersection a&b
union a|b
complement ~a
difference a & (~ b)
*/
```

6.2 Template

```
#include <bits/stdc++.h>
#define fastio ios::sync_with_stdio(0); cin.tie(nullptr); cout.tie(nullptr);
#define ll long long
#define endl '\n'
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define pb push_back

using namespace std;

const int oo = 1e9;
const ll OO = 1e18;

typedef vector<int> vi;
typedef vector<ll> vll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<pll> vpll;
typedef vector<pii> vpii;

signed main(){fastio
    //code
    //comes
    //here

    return 0;
}
```

6.3 Math Misc

```
//Handbook
bool prime(int n) {
    if (n < 2) return false;
    for (int x = 2; x*x <= n; x++) {
        if (n%x == 0) return false;
    }
    return true;
}
```

6.4 Cpp Functions

```
//Binary search

//lower_bound(arr, arr + n, x) returns a pointer to the first array element
//whose value is at least x
//upper_bound(arr, arr + n, x) returns a pointer to the first array element
//whose value is larger than x.
//equal_range(arr, arr + n, x) returns both above pointers.

//Number of elements whose value is x
auto r = equal_range(array, array+n, x);
cout << r.second-r.first << "\n";

//Data structures
```

```
//Remove only one instance of a repeated number in a multiset
multiset<int> s;
s.erase(s.find(x));

//Checks if a key exists in a map
map<string, int> m;
m.count("OI");

//largest element in a set
set<int> s;
auto it = s.end(); it--;
cout << *it << "\n";

//Priority queue min-heap
priority_queue<int, vector<int>, greater<int>> q;
```

6.5 Outros Notebooks

<https://github.com/Tiagosf00/Competitive-Programming>

<https://github.com/brunomaletta/biblioteca>

<https://github.com/scanof/Competitive-Programming-Notebook>